

Prueba de Práctica 3: Conceptos Generales de Programación Avanzada

Programación Avanzada — Evaluación Autocontenida Integrada

Parte 1: Preguntas de Selección Múltiple

Pregunta 1. Iterable vs Iterador en Python.

¿Cuál de las siguientes afirmaciones describe con precisión la diferencia teórica y práctica entre un objeto iterable y un objeto iterador en Python?

- a) Un iterable implementa obligatoriamente el método `__next__()`, mientras que un iterador solo implementa `__iter__()`.
- b) Cualquier objeto sobre el cual se pueda llamar `iter()` es un iterable, el cual retorna un objeto iterador. El iterador es el encargado de mantener el estado de la iteración y entregar los elementos uno a uno mediante `next()`.
- c) Un iterador puede ser reiniciado y recorrido múltiples veces desde el principio en cualquier momento, mientras que un iterable se consume y queda inutilizable tras el primer ciclo.
- d) El intérprete de Python no permite convertir un iterable en un iterador en tiempo de ejecución.
- e) No existe diferencia práctica ni semántica entre ambos conceptos en Python 3.

Pregunta 2. Comportamiento de la función incorporada `zip()`.

Considere la función integrada de Python `zip(*iterables)`. Si le entregamos como parámetros dos iterables de tamaños significativamente distintos (por ejemplo, una lista de 100 elementos y otra de 3 elementos), ¿cuál es el comportamiento por defecto de la función?

- a) Lanza una excepción de tipo `ValueError` debido a la inconsistencia de dimensiones.
- b) Rellena automáticamente el iterable más corto con valores `None` hasta alcanzar el tamaño del más largo.
- c) Retorna un iterador de tuplas que se detiene de forma limpia tan pronto como el iterable más corto se agota, ignorando los elementos restantes del más largo.
- d) Entra en un bucle infinito intentando calcular la correspondencia de índices virtuales.
- e) Levanta la excepción `IndexError` en la última tupla generada.

Pregunta 3. Coordinación mediante Eventos.

El objeto `threading.Event` permite sincronizar hilos en base a banderas booleanas internas gestionadas de forma segura. ¿Qué ocurre con un hilo que invoca el método `evento.wait()` si la bandera interna del evento se encuentra actualmente en estado `False`?

- a) El método retorna inmediatamente **False** y el hilo continúa ejecutándose de manera asíncrona.
- b) El hilo levanta de forma inmediata un error del tipo **TimeoutException**.
- c) El hilo se suspende y se bloquea de forma indefinida en esa línea de instrucción hasta que otro hilo llame a **evento.set()**.
- d) El sistema operativo elimina el hilo bloqueado para liberar recursos.
- e) La bandera interna del evento cambia automáticamente a **True** para evitar que el hilo espere.

Pregunta 4. Uso correcto de *QThread* para tareas concurrentes.

¿Cuál es la forma recomendada en PyQt5 para estructurar una tarea concurrente que reporta avances intermedios a la interfaz usando la clase *QThread*?

- a) Sobrescribir el método **run()** de una subclase de *QThread*, definir allí señales como atributos de clase y emitir dichas señales con los progresos intermedios para que la GUI los conecte a sus widgets.
- b) Invocar directamente a **QThread.start()** pasándole una función lambda con la lógica de dibujo de los widgets.
- c) Ejecutar un bucle **while** infinito dentro de la GUI y llamar a **QThread.sleep()** para forzar la actualización gráfica.
- d) Instanciar *QThread* y modificar los atributos privados de la ventana principal de forma directa en el código del hilo.
- e) El uso de *QThread* está obsoleto en PyQt5, debiendo usarse exclusivamente la biblioteca externa **asyncio**.

Pregunta 5. Gestión de Eventos a bajo nivel en Qt.

En PyQt5, cuando se presiona un botón del mouse sobre una ventana, Qt genera un objeto de tipo *QMouseEvent* y lo propaga a través de un método de evento específico. Si deseamos interceptar y modificar esta interacción a bajo nivel, ¿qué método debemos sobrescribir en nuestra clase de ventana personalizada?

- a) **clickEvent(self, event)**
- b) **mousePressEvent(self, event)**
- c) **on_mouse_click(self, event)**
- d) **captureMouse(self, event)**
- e) **mouse_trigger(self, event)**

Parte 2: Preguntas de Desarrollo

Pregunta 6. Protocolo de Iteración y Backtracking (Materia: Excepciones e Iteradores). Implemente un sistema básico para gestionar el historial de navegación de un usuario en un editor de texto que admita la acción de deshacer (backtracking).

- a) Diseñe una clase `HistorialAcciones` que almacene una lista de cadenas de texto (las acciones realizadas).
- b) Implemente el protocolo de iteración manual para la clase de modo que permita recorrer el historial en orden inverso (del cambio más nuevo al más antiguo) usando un iterador interno personalizado llamado `IteradorInverso`.
- c) El iterador debe soportar el método especial `__next__` de forma manual y levantar `StopIteration` cuando se complete la secuencia.

Pregunta 7. Procesamiento Funcional de Logs (Materia: Programación Funcional). Considere una lista de líneas de un archivo de log de servidor con el formato: "`[FECHA]` `[CATEGORIA]` MENSAJE". Ejemplo: "`[2026-05-27]` `[ERROR]` Connection timed out". Implemente un pipeline de programación funcional que reciba esta lista de cadenas y:

- Filtre las líneas de log que sean exclusivamente de categoría "`[ERROR]`".
- Extraiga (mapee) únicamente el mensaje del error (es decir, elimine la fecha y la categoría).
- Cuente la cantidad total de errores encontrados utilizando reducción o sumatoria funcional.
- Requisito: Implemente todo el flujo encadenando `'filter'`, `'map'` y `'reduce'` en una única instrucción/línea de código.

Pregunta 8. Coordinación en Grupo usando Eventos (Materia: Threading y Concurrencia).

En una aplicación de renderizado 3D de alta exigencia, se necesita iniciar el cálculo de un frame complejo solo cuando todos los hilos trabajadores de renderizado (3 hilos en total) estén completamente inicializados y listos. Implemente un script donde:

- Un evento global llamado `evento_inicio` mantenga a los trabajadores esperando en su estado inicial.
- Cada trabajador (hilo) realice una inicialización simulada (tiempo aleatorio entre 0.1 y 0.5 segundos) y luego deba bloquearse hasta que se dé la señal global de inicio de cálculo.
- El hilo principal controle la coordinación: espere a que todos los hilos confirmen estar preparados y posteriormente active el evento global llamando a `set()` para iniciar el cálculo simultáneo.

Pregunta 9. Concurrency en Interfaces con *QThread* (Materia: GUI PyQt5).

En un visor de procesos, se desea presionar un botón para iniciar una tarea pesada (cálculo de 0 a 100 con demoras intermitentes) y mostrar el progreso en tiempo real usando un componente `QProgressBar` de la interfaz gráfica sin bloquear la ventana durante el proceso.

- a) Diseñe una clase `WorkerThread` que herede de `QThread` y defina una señal llamada `progreso_actualizado` de tipo `int`.
- b) Implemente el método `run()` de modo que itere de 0 a 100 con pausas de 0.05 segundos, emitiendo la señal en cada incremento.
- c) Implemente la clase `VentanaProceso` con un botón para iniciar la tarea y un `QProgressBar` para visualizar los avances interconectando la señal de progreso.

Solucionario

Respuestas de Selección Múltiple

Pregunta 1: b)

Explicación: En Python, un iterable es cualquier objeto capaz de retornar sus elementos uno a uno cuando se le solicita (implementa `__iter__()`). El iterador es el objeto retornado por `__iter__()` y se encarga de realizar el recorrido manteniendo la posición actual (implementa `__next__()` e `__iter__()`). Una vez consumido el iterador, no se puede reiniciar; se debe solicitar un nuevo iterador al iterable.

Pregunta 2: c)

Explicación: Por especificación de diseño de Python, la función `zip()` empareja elementos posicionales de los iterables entregados. Cuando el más corto de los iterables se queda sin elementos, `zip()` detiene la generación de tuplas levantando internamente la señal de término estándar de forma transparente. Para emparejar rellenando elementos vacíos, se debe recurrir a `itertools.zip_longest()`.

Pregunta 3: c)

Explicación: Los objetos de tipo **Event** manejan una bandera booleana interna (inicialmente **False**). El método `wait()` comprueba dicha bandera. Si es **False**, el hilo invocador se bloquea en ese punto. El bloqueo se levanta únicamente cuando otro hilo de control cambia el estado del evento llamando a `set()`, lo cual despierta simultáneamente a todos los hilos que se encontraban suspendidos en llamadas a `wait()`.

Pregunta 4: a)

Explicación: La arquitectura limpia en PyQt5 consiste en derivar de **QThread** y definir la tarea pesada dentro del método sobrescrito `run()`. Dentro de este método, para reportar cambios (como porcentajes de avance), se emite una señal personalizada. El hilo principal de la GUI, al crear la instancia del hilo secundario, conecta esta señal a un slot (por ejemplo, el método `setValue` de un **QProgressBar**). De esta forma, el paso de datos cruza de un hilo a otro de manera segura y sincronizada por el framework.

Pregunta 5: b)

Explicación: Para capturar clics de ratón a bajo nivel sobre componentes de Qt que no sean botones interactivos dedicados, se debe redefinir el método de evento nativo `mousePressEvent(self, event)` en la clase widget correspondiente. Al sobrescribirlo, podemos extraer las coordenadas exactas de la pulsación utilizando el objeto de evento recibido por parámetro.

Soluciones de Desarrollo

Pregunta 6 (Protocolo de Iteración y Backtracking):

```
class IteradorInverso:
    def __init__(self, lista_datos):
        self.datos = lista_datos
        self.indice_actual = len(lista_datos) - 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.indice_actual < 0:
            raise StopIteration

        # Obtenemos el elemento en el índice actual y retrocedemos
        elemento = self.datos[self.indice_actual]
        self.indice_actual -= 1
        return elemento

class HistorialAcciones:
    def __init__(self):
        self.acciones = []

    def agregar_accion(self, accion: str):
        self.acciones.append(accion)

    def __iter__(self):
        # Retorna el iterador personalizado inverso
        return IteradorInverso(self.acciones)

# Ejemplo de uso
historial = HistorialAcciones()
historial.agregar_accion("Crear archivo")
historial.agregar_accion("Escribir Hola")
historial.agregar_accion("Negrita en Hola")

print("Recorriendo historial para deshacer (Backtracking):")
for accion in historial:
    print(f"Deshaciendo: {accion}")
    # Imprime en orden: Negrita en Hola, Escribir Hola, Crear archivo
```

Pregunta 7 (Procesamiento Funcional de Logs):

```
from functools import reduce

# Logs de prueba
lineas_log = [
    "[2026-05-27] [INFO] Server started",
    "[2026-05-27] [ERROR] Connection timed out",
    "[2026-05-27] [WARNING] Low memory space",
    "[2026-05-27] [ERROR] Database authentication failed"
]

# Pipeline funcional en una sola línea
# 1. Filtra las líneas que contienen '[ERROR]'
# 2. Mapea para extraer el mensaje quitando el prefijo de fecha y categoría
# 3. Reduce para sumar 1 por cada error procesado
total_errores = reduce(lambda acc, _: acc + 1, map(lambda line: line.split("] [ERROR] "),
                                                    lineas_log))

print(f"Cantidad total de errores procesados: {total_errores}") # Imprime: 2
```

Pregunta 8 (Coordinación en Grupo usando Eventos):

```
import threading
import time
import random

def trabajador_render(id_worker, evento_inicio):
    # Simulación de inicialización de texturas y geometría
    delay = random.uniform(0.1, 0.5)
    time.sleep(delay)
    print(f"[Worker {id_worker}] Inicializado en {delay:.2f}s. Esperando señal global...")

    # El hilo se bloquea hasta que la bandera del evento sea True
    evento_inicio.wait()

    # Procesamiento una vez dada la señal
    print(f"[Worker {id_worker}] >>> ¡Señal recibida! Renderizando frame...")
    time.sleep(0.2)
    print(f"[Worker {id_worker}] <<< Renderizado finalizado.")

# Evento de control compartido
evento_arranque = threading.Event()

# Creación de hilos
workers = []
```

```

for i in range(1, 4):
    hilo = threading.Thread(
        target=trabajador_render,
        args=(i, evento_arranque)
    )
    workers.append(hilo)
    hilo.start()

# Simulación de espera en el hilo principal
print("[Principal] Hilo principal preparando buffers globales...")
time.sleep(0.8) # Damos un margen de tiempo para que todos los trabajadores se listen

print("[Principal] Todos los recursos listos. ¡Iniciando cálculo de renderizado!")
# Activamos el evento. Esto liberará simultáneamente a los 3 trabajadores bloqueados
evento_arranque.set()

for w in workers:
    w.join()

print("[Principal] Frame renderizado con éxito.")

```

Pregunta 9 (Concurrencia en Interfaces con QThread):

```
import sys
import time
from PyQt5.QtWidgets import (QApplication, QWidget, QVBoxLayout,
                             QPushButton, QProgressBar)
from PyQt5.QtCore import QThread, pyqtSignal

# a) Subclase QThread para cómputo en segundo plano
class WorkerThread(QThread):
    progreso_actualizado = pyqtSignal(int) # Flujo de datos hacia la GUI

    # b) Sobrescribir método run
    def run(self):
        for i in range(101):
            time.sleep(0.05) # Simula cálculo exigente o lectura
            self.progreso_actualizado.emit(i) # Envío seguro

# c) Interfaz Gráfica Principal
class VentanaProceso(QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Control de Hilos")
        self.resize(300, 100)

        layout = QVBoxLayout()

        self.progreso = QProgressBar()
        self.progreso.setValue(0)
        layout.addWidget(self.progreso)

        self.btn_iniciar = QPushButton("Iniciar Proceso")
        self.btn_iniciar.clicked.connect(self.comenzar_tarea)
        layout.addWidget(self.btn_iniciar)

        self.setLayout(layout)

    def comenzar_tarea(self):
        # Crear la instancia del hilo trabajador
        self.hilo = WorkerThread()

        # Conectar la señal al slot nativo del QProgressBar
```

```
self.hilo.progreso_actualizado.connect(self.progreso.setValue)

# Desactivar botón temporalmente para evitar ejecuciones duplicadas
self.hilo.started.connect(lambda: self.btn_iniciar.setEnabled(False))
self.hilo.finished.connect(lambda: self.btn_iniciar.setEnabled(True))

# Iniciar el hilo auxiliar de forma asíncrona
self.hilo.start()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana = VentanaProceso()
    ventana.show()
    sys.exit(app.exec_())
```